운영체제 요약 정리

핵심 개념 및 시험 대비 학습 노트

목차

정리 범위와 원문 확인 사항 **3**

전체 학습 내용 개요 **4**

장별 요약 및 상세 정리 **6**

제1장. 서론 6

제2장. 운영체제 구조 18

제7부 도입. 보안과 보호 19

비교·공식·절차 모음 **21**

주요 개념 복습 **22**

개념 색인 **23**

최종 학습 점검표 **24**

정리 범위와 원문 확인 사항

제공된 615쪽 PDF의 모든 페이지를 대상으로 목차·본문·표·그림·수식·알고리즘·각주와 장말 자료를 검토한 학습 자료다. 실제 본문이 있는 제1장과 제3~15장, 제1·3·4·5·6·7부 도입을 정리한다. 원본 책 전체가 제공된 것은 아니므로, 책 전체의 완전한 요약으로 표시하지 않는다.

제공된 PDF의 전체 쪽수: 615

확인 가능한 범위: PDF 1~5쪽은 원본 목차, 6~615쪽은 제공된 도입 및 본문이다. 인쇄 1~679쪽 중 실제 제공된 610쪽에 해당한다. 실제 장별 범위는 아래 표와 각 장의 출처를 따른다.

원문 확인 범위와 제한

- 제2장 운영체제 구조 본문은 제공되지 않았다. 목차에서 제목·절 이름·시작 위치만 확인할 수 있어 제2장에는 미제공 안내만 남겼다.
- 제16~21장, 부록과 원본 찾아보기는 목차에만 있고 실제 본문이 없다. 이들 장의 학습 내용을 새로 작성하지 않았다.
- 인쇄 2, 60~116, 174, 280, 282, 318, 348, 382, 384, 490, 578, 616, 654쪽은 제공 파일에서 확인되지 않는다. 장 경계의 누락도 빈 페이지라고 추정하지 않았다.
- 마지막 PDF 615쪽은 제7부 보안과 보호의 도입이다. 그 도입에서 확인되는 정의만 정리하며, 뒤의 보안·보호 장 내용을 읽은 것으로 표시하지 않는다.
- 원문은 이미지 스캔이다. OCR은 독서 보조로 사용하고, 수록한 코드·수식·수치 예제와 중요한 그림은 원문 이미지로 대조하거나 원문 이미지를 그대로 보존했다. 의심되는 표기는 원문 확인 필요, 확인할 수 없는 내용은 원문에서 확인되지 않음으로 표시한다.
- 장말 연습문제는 점검하는 개념을 확인하는 데 사용했다. 원문에 없는 새 문제나 풀이·실행 결과를 만들지 않았으며, 외부 자료로 설명을 보충하지 않았다.

출처의 "책"은 원문에 인쇄된 쪽수, "원문 PDF"는 제공 파일의 첫 쪽을 1로 세는 위치다. 이 자료의 목차·쪽번호·색인은 생성된 PDF의 실제 쪽수를 사용한다. 원본 목차는 로마자 xix~xxiii로 인쇄돼 있어 본문 인쇄 쪽수와 구별한다.

전체 학습 내용 개요

제1부. 개관

출처: 책 1쪽 / 원문 PDF 6쪽

운영체제는 사용자와 하드웨어 사이의 중개자이며, 프로그램을 편리하고 효율적으로 실행할 환경을 제공한다. 하드웨어를 관리하면서 시스템의 정상 동작을 지키고, 설계 목표와 각 구성 부분의 입력·출력·동작을 명확히 해야 한다.

제3부. 프로세스 동기화

출처: 책 281쪽 / 원문 PDF 226쪽

병행 또는 병렬로 실행되는 스레드가 데이터를 공유할 때 접근을 제어하지 않으면 경쟁 조건으로 값이 손상될 수 있다. 동기화 도구는 공유 데이터 접근을 제어하지만, 잘못 사용하면 교착 상태나 성능 저하를 일으킬 수 있다.

제4부. 메모리 관리

출처: 책 383쪽 / 원문 PDF 324쪽

실행 중인 프로그램과 접근하는 데이터의 적어도 일부는 메인 메모리에 있어야 한다. 여러 프로세스를 메모리에 유지하는 관리 기법의 효과는 상황에 따라 다르며, 선택은 하드웨어 설계와 지원의 영향을 받는다.

제5부. 저장장치 관리

출처: 책 489쪽 / 원문 PDF 429쪽

대용량 저장장치는 파일과 데이터를 영구 저장한다. 장치는 전송 단위·접근 방식·동기성·공유 여부·읽기와 쓰기·속도가 다양하다. 입출력 서브시스템은 이를 제어하면서 단순한 인터페이스를 제공하고, 병목을 줄이도록 입출력을 최적화한다.

제6부. 파일 시스템

출처: 책 577쪽 / 원문 PDF 516쪽

파일은 작성자가 정의한 관련 정보의 모음이다. 파일 시스템은 파일을 물리적 저장장치에 매핑하고 사용자·프로그램의 접근과 조작 방법을 제공한다. 느릴 수 있는 물리 장치의 효율적인 접근과 공유·원격 접근 요구를 함께 고려한다.

제공된 장별 학습 범위

제1장. 서론

출처: 책 3-59쪽 / 원문 PDF 7-63쪽

- 운영체제를 하드웨어 자원 관리자이자 프로그램 실행 환경으로 이해한다. 컴퓨터 구성·인터럽트·저장장치·실행 모드에서 출발해 자원 관리, 보호와 보안, 가상화, 분산 시스템, 자료구조와 계산 환경으로 범위를 넓힌다.

제2장. 운영체제 구조

출처: 원문 PDF 1쪽

- 제공 파일의 목차에는 제2장이 있지만 해당 본문은 제공되지 않았다. 본문 지식은 원문에서 확인되지 않음.

제7부 도입. 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

- 제공된 마지막 도입 페이지에서 보안과 보호의 역할을 구별한다.

장	원문 범위
제1장. 서론	출처: 책 3-59쪽 / 원문 PDF 7-63쪽
제2장. 운영체제 구조	출처: 원문 PDF 1쪽
제7부 도입. 보안과 보호	출처: 책 679쪽 / 원문 PDF 615쪽

장별 요약 및 상세 정리

제1장. 서론

출처: 책 3-59쪽 / 원문 PDF 7-63쪽

학습 주제

- 운영체제를 하드웨어 자원 관리자이자 프로그램 실행 환경으로 이해한다. 컴퓨터 구성·인터럽트·저장장치·실행 모드에서 출발해 자원 관리, 보호와 보안, 가상화, 분산 시스템, 자료구조와 계산 환경으로 범위를 넓힌다.

설명의 흐름

1. 컴퓨터 구성과 인터럽트 → 저장장치와 처리기 구조 → 운영체제의 제어·자원 관리 → 보호·가상화·분산 환경 → 커널 자료구조와 공개 소스 학습

1.1 운영체제가 할 일

출처: 책 4-6쪽 / 원문 PDF 8-10쪽

운영체제는 컴퓨터 하드웨어를 관리하고 응용 프로그램의 실행 기반을 제공하며 사용자와 하드웨어 사이를 중재하는 소프트웨어이다. 하드웨어는 CPU·메모리·입출력 장치로 기본 계산 자원을 제공한다. 응용 프로그램은 이 자원을 사용해 사용자의 문제를 해결하며, 운영체제는 여러 사용자와 프로그램 사이의 하드웨어 사용을 제어하고 조정한다.

관점	핵심 역할과 조건
사용자 관점	개인용 컴퓨터에서는 사용의 편리함이 주요 목표이다. 모바일 장치는 터치·음성 인터페이스를 제공한다. 내장형 컴퓨터는 사용자 개입 없이 동작하기도 한다.
시스템 관점	자원 할당자로서 CPU 시간·메모리·저장공간·입출력 장치의 상충하는 요청을 효율적이고 공정하게 처리한다. 제어 프로그램으로서 부적절한 사용을 방지한다.

운영체제에 포함되는 요소에는 보편적으로 합의된 경계가 없다. 원문은 항상 실행되는 프로그램인 **커널**, 관리에 쓰이는 시스템 프로그램, 개발자에게 추가 서비스를 제공하는 미들웨어를 구별한다. 커널과 관련된 프로그램이라도 반드시 커널의 일부인 것은 아니다. 모바일 운영체제에는 데이터베이스·멀티미디어·그래픽 지원 프레임워크도 포함된다.

1.2 컴퓨터 시스템의 구성

출처: 책 7쪽 / 원문 PDF 11쪽

범용 컴퓨터는 하나 이상의 CPU, 공유 메모리, 공통 버스로 연결된 장치 컨트롤러로 구성된다. 컨트롤러는 특정 유형의 장치를 담당하고 로컬 버퍼와 특수 목적 레지스터를 가진다. **장치 드라이버**는 컨트롤러의 특성을 알고 운영체제의 다른 부분에 일관된 인터페이스를 제공한다. CPU와 장치 컨트롤러가 병렬로 작동하므로 공유 메모리 접근은 동기화되어야 한다.

1.2.1 인터럽트

출처: 책 7-12쪽 / 원문 PDF 11-16쪽

인터럽트는 하드웨어가 CPU의 처리가 필요한 이벤트를 알리는 핵심 수단이다. 드라이버가 컨트롤러 레지스터에 값을 적재하면 컨트롤러가 작업을 시작하고, 완료 후 인터럽트로 알린다. CPU는 하던 실행을 중단하고 서비스 루틴으로 제어를 옮긴 뒤 중단된 연산을 재개한다.

인터럽트 처리의 기본 순서

1. CPU는 명령어 실행을 마칠 때 인터럽트 요청 라인을 검사한다.
2. 인터럽트 번호를 인터럽트 벡터의 인덱스로 사용해 해당 핸들러로 이동한다.
3. 핸들러는 변경될 상태를 저장하고 원인을 확인하여 필요한 처리를 수행한다.
4. 상태를 복원하고 인터럽트 복귀 명령으로 이전 실행을 재개한다.

인터럽트 벡터는 서비스 루틴 주소의 배열이다. 중간 검색 루틴을 거치지 않고 적절한 루틴을 호출하여 처리 시간을 줄인다. 장치 수가 벡터 항목 수보다 많으면 각 항목에 핸들러 리스트를 연결하는 **인터럽트 체인**을 사용하고 요청을 처리할 핸들러를 찾을 때까지 호출한다.

구분	원문의 설명
마스크 불가능 인터럽트	복구할 수 없는 메모리 오류 같은 이벤트를 위해 예약한다.
마스크 가능 인터럽트	중요한 명령 시퀀스 실행 전에 잠시 끌 수 있으며 장치의 서비스 요청에 사용된다.
우선순위 레벨	낮은 우선순위 처리를 연기하고 높은 우선순위 인터럽트가 낮은 우선순위 실행을 선점하게 한다.

인터럽트 서비스는 빈번하고 시간에 민감하므로 효율적이어야 한다. 복귀 주소와 변경되는 레지스터 상태를 보존하지 않으면 원래 실행을 올바르게 재개할 수 없다.

1.2.2 저장장치 구조

출처: 책 12-15쪽 / 원문 PDF 16-19쪽

실행할 프로그램은 먼저 **메인 메모리**에 적재되어야 한다. 일반적인 메인 메모리는 DRAM으로 구현되는 휘발성 저장장치이다. 전원을 켤 때 운영체제를 적재하는 부트스트랩 프로그램은 RAM에 영구 보관할 수 없으므로 EEPROM 등 비휘발성 펌웨어를 사용한다.

적재(load)는 메인 메모리의 바이트 또는 워드를 CPU 레지스터로 옮기고, 저장(store)은 반대로 옮긴다. 명령 실행은 명령 인출·해독·피연산자 인출·연산·결과 저장으로 진행될 수 있다. 메모리는 주소가 생성된 방법이나 그 내용이 명령인지 데이터인지를 구별하지 않고 주소의 연속으로 취급한다.

메인 메모리는 모든 프로그램과 데이터를 영구 저장하기에는 작고 전원 손실 시 내용을 잃는다. 보조저장장치는 이를 확장해 대량의 데이터를 영구 보존한다. HDD와 NVM은 프로그램이 메모리에 적재되기 전 저장소이며, 처리 중 데이터의 원천과 목적지가 되기도 한다.

구분	원문의 범위
휘발성 저장장치	전원 제거 시 내용을 잃는다. 안전하게 보존할 데이터는 비휘발성 장치에 기록해야 한다.

구분	원문의 범위
기계적 비휘발성 저장장치	HDD·광 디스크·자기 테이프 등. 일반적으로 전기적 장치보다 용량이 크고 바이트당 비용이 낮다.
전기적 비휘발성 저장장치(NVM)	플래시·FRAM·NRAM·SSD 등. 일반적으로 더 빠르지만 비싸고 용량이 작다.
3차 저장장치	백업·드물게 쓰는 데이터·장기 보관을 위한 느리고 큰 저장장치이다.

저장 계층은 속도·용량·비용·휘발성을 함께 고려한다. 작고 빠른 메모리는 CPU에 가깝게 두며, 접근 시간이나 전송 속도 차이가 큰 두 수준 사이에는 캐시를 둘 수 있다. 원문은 1바이트=8비트, 워드를 구조 본연의 데이터 단위로 설명한다. 저장장치의 바이트 단위와 네트워킹의 비트 단위를 구별한다.

그림 1.6 저장장치 계층 구조

그림·표 출처: 책 14쪽 / 원문 PDF 18쪽

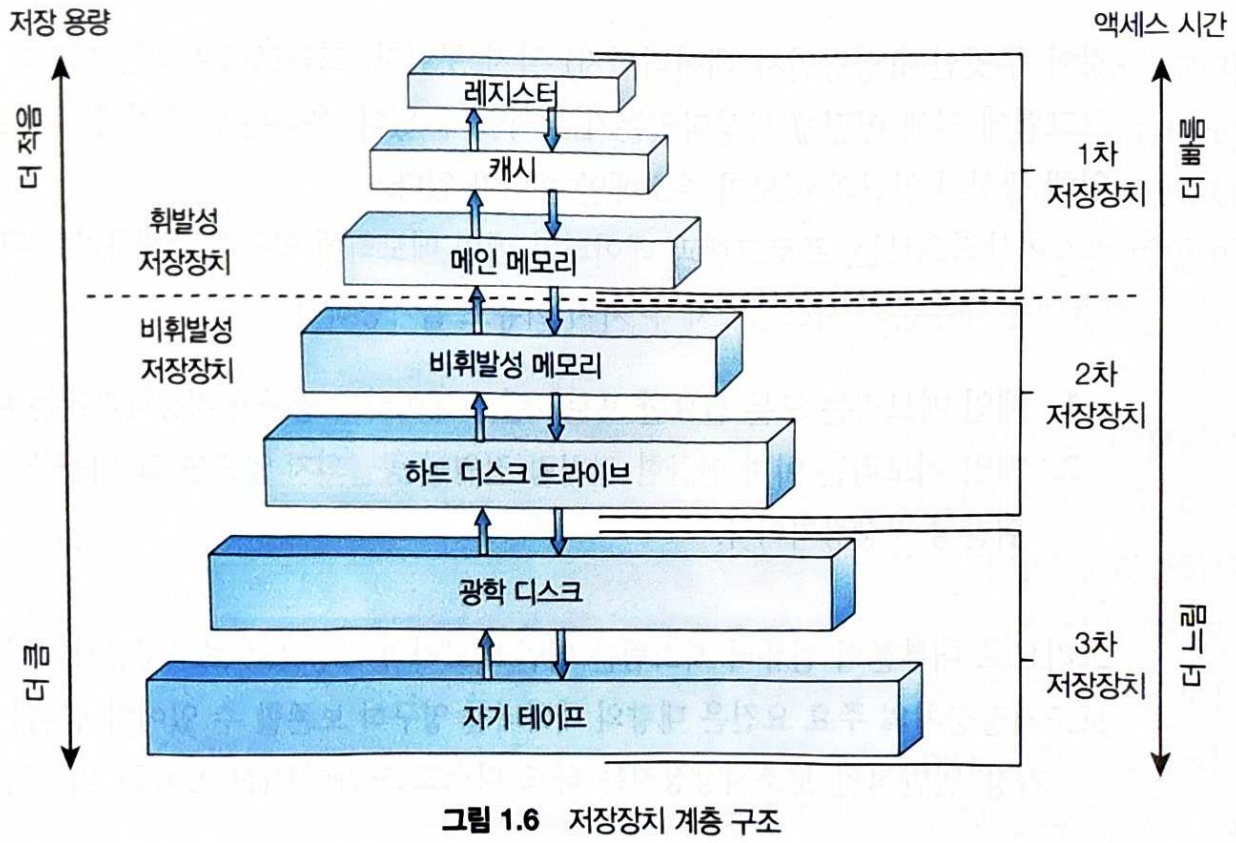


그림 1.6 저장장치 계층 구조

그림 1.6 저장장치 계층 구조

레지스터에서 자기 테이프로 내려가며 저장 용량은 커지고 접근 시간은 길어진다. 휘발성·비휘발성 및 1·2·3차 저장장치의 경계를 원문 그대로 보존하였다.

1.2.3 입출력 구조

출처: 책 15-16쪽 / 원문 PDF 19-20쪽

소량 데이터에는 인터럽트 구동 입출력이 적합하지만 대량 전송에서 바이트마다 인터럽트를 발생시키면 오버헤드가 커진다. **직접 메모리 접근(DMA)**은 버퍼·포인터·입출력 카운트를 설정한 후 컨트롤러가 CPU 개입 없이 메모리와 장치 버퍼 사이에서 블록 전체를 전송하게 한다. 블록 전송이 끝날 때 인터럽트를 발생시키므로 그동안 CPU는 다른 작업을 할 수 있다. 스위치 구조는 공유 버스 경쟁 없이 여러 구성요소의 동시 통신을 허용하여 DMA를 더 효과적으로 만들 수 있다.

1.3.1 단일 처리기 시스템

출처: 책 16-17쪽 / 원문 PDF 20-21쪽

원문은 단일 처리 코어를 가진 범용 CPU가 하나인 경우를 단일 처리기 시스템으로 설명한다. 디스크·키보드·그래픽 컨트롤러의 전용 처리기는 제한된 명령어만 실행하고 사용자 프로세스를 실행하지 않는다. 그러한 전용 처리기가 있다고 해서 다중 처리기 시스템이 되는 것은 아니다. 코어는 명령을 실행하고 로컬 데이터용 레지스터를 갖는 기본 계산 단위이다.

1.3.2 다중 처리기 시스템

출처: 책 17-20쪽 / 원문 PDF 21-24쪽

다중 처리기 시스템은 여러 처리기가 버스와 메모리·주변 장치 등을 공유한다. 처리기를 늘리면 처리량이 증가하지만 협력 오버헤드와 공유 자원 경합 때문에 N개 처리기의 속도 향상이 반드시 N배가 되지는 않는다. **대칭형 다중 처리(SMP)**에서는 각 CPU가 운영체제 기능과 사용자 프로세스를 포함한 모든 작업을 수행한다.

각 CPU는 자체 레지스터와 로컬 캐시를 가지며 물리 메모리를 공유한다. 한 CPU가 유휴이고 다른 CPU가 과부하인 상태를 줄이려면 작업과 자료구조를 적절히 공유해야 한다. **다중 코어**는 한 칩 안에 여러 계산 코어를 두는 방식이다. 칩 내부 통신의 효율과 낮은 전력 소모가 이점이며 로컬 캐시와 공유 캐시를 결합할 수 있다. 운영체제에는 N개 코어가 N개 CPU처럼 보인다.

공유 버스 경합이 확장성의 병목이 되면 **NUMA** 구조에서 각 CPU 또는 CPU 그룹에 로컬 메모리를 둔다. 모든 CPU는 하나의 물리 주소 공간을 공유하지만 로컬 메모리보다 원격 메모리 접근 지연이 크다. 운영체제의 스케줄링과 메모리 관리가 이 단점을 줄여야 한다. 블레이드 서버는 각 보드가 독립적으로 부팅하고 자신의 운영체제를 실행한다는 점에서 전통적인 공유형 다중 처리기와 다르다.

1.3.3 클러스터형 시스템

출처: 책 20-22쪽 / 원문 PDF 24-26쪽

클러스터형 시스템은 둘 이상의 독립적인 노드를 네트워크로 연결한 약결합 시스템이다. 일반적으로 저장장치를 공유하고 LAN이나 고속 상호 연결망을 사용한다. 노드가 서로를 감시하다가 고장 노드의 저장장치 소유권을 넘겨받고 응용을 다시 실행함으로써 높은 가용성을 제공한다.

구성	동작
비대칭형 클러스터	다른 호스트가 응용을 실행하는 동안 긴급 대기 호스트는 감시만 하다가 고장 시 활성 서버가 된다.
대칭형 클러스터	여러 호스트가 응용을 실행하며 서로 감시한다. 모든 하드웨어를 이용하지만 여러 응용이 실행 가능해야 효율적이다.

남은 하드웨어 수준에 비례해 서비스를 계속하는 것이 우아한 성능 저하이며, 결함허용 시스템은 단일 구성요소 오류에도 계속 작동하도록 감지·진단·수정 기법을 둔다. 고성능 계산에 클러스터를 사용하려면 응용을 병렬화하여 각 노드가 일부 문제를 해결한 뒤 결과를 합쳐야 한다. 공유 데이터의 병렬 접근에는 충돌을 막는 접근 제어·잠금이 필요하며 원문은 분산 잠금 관리자(DLM)를 예로 든다. SAN은 여러 호스트가 여러 저장장치에 연결되도록 한다.

1.4 운영체제의 작동

출처: 책 23-24쪽 / 원문 PDF 27-28쪽

부팅 시 펌웨어의 **부트스트랩 프로그램**이 CPU 레지스터·컨트롤러·메모리를 초기화하고 커널을 찾아 메모리에 적재한다. 커널이 실행되면 서비스를 제공하고, 일부 서비스는 부팅 때 적재되어 계속 실행되는 시스템 데몬이 맡는다. 원문은 Linux의 systemd를 다른 데몬을 시작하는 시스템 프로그램으로 제시한다.

실행할 일이 없으면 시스템은 이벤트를 기다린다. 하드웨어 인터럽트 외에 **트랩(예외)**은 0으로 나누기·유효하지 않은 메모리 접근 등의 오류나 사용자 프로그램의 특정 요청으로 발생하는 소프트웨어 인터럽트이다. 사용자 요청은 시스템 콜을 통해 운영체제 서비스를 요구한다. 원문의 Hadoop 보충 설명은 분산 파일 시스템·YARN의 자원 및 작업 관리·MapReduce의 병렬 처리가 클러스터에서 결합됨을 보여 준다.

1.4.1 다중 프로그래밍과 다중 태스킹

출처: 책 24-26쪽 / 원문 PDF 28-30쪽

다중 프로그래밍은 여러 프로세스를 메모리에 유지하여 한 프로세스가 입출력 등을 기다릴 때 다른 프로세스를 실행한다. 실행 가능한 일이 있는 동안 CPU를 유휴로 두지 않아 이용률을 높이는 것이 목적이다. **다중 태스킹**은 이를 확장하여 프로세스 전환을 자주 수행함으로써 사용자에게 빠른 응답을 제공한다.

여러 프로세스를 동시에 유지하려면 메모리 관리가, 준비된 여러 프로세스 중 다음 대상을 고르려면 CPU 스케줄링이 필요하다. 가상 메모리는 일부만 메모리에 적재된 프로세스도 실행하게 하여 논리 메모리를 물리 메모리와 분리한다. 파일·저장장치 관리, 자원 보호, 통신·동기화와 교착 상태 처리도 여러 프로세스의 질서 있는 실행과 연결된다.

1.4.2 이중 모드와 다중 모드 운용

출처: 책 26-28쪽 / 원문 PDF 30-32쪽

운영체제와 사용자 코드가 자원을 공유하므로 하드웨어는 **사용자 모드**와 **커널 모드**를 구별한다. 원문의 모드 비트 예에서는 커널 모드가 0, 사용자 모드가 1이다. 부팅은 커널 모드에서 시작하고 사용자 프로그램으로 제어를 넘길 때 사용자 모드로 전환한다. 인터럽트·트랩·시스템 콜이 발생하면 하드웨어가 커널 모드로 바꾼다.

특권 명령은 커널 모드에서만 허용한다. 입출력 제어·타이머 관리·인터럽트 관리 등이 그 예이다. 사용자 모드에서 실행하려 하면 불법 명령으로 트랩을 발생시켜 운영체제로 제어를 넘긴다. 모드는 둘 이상으로 확장될 수 있으며 원문은 Intel 보호 링, ARMv8의 여러 모드, 가상 머신 관리자를 위한 모드를 소개한다.

시스템 콜의 제어 흐름

1. 사용자 프로세스가 운영체제의 작업 수행을 요청한다.
2. 트랩과 인터럽트 벡터를 통해 서비스 루틴으로 제어가 이동하고 커널 모드가 된다.
3. 커널이 호출 종류와 인수의 정확성·합법성을 검사하여 요청을 수행한다.
4. 사용자 모드로 바꾸고 시스템 콜 다음 명령으로 복귀한다.

프로그램이 불법 명령이나 자신의 영역 밖 메모리에 접근하면 하드웨어가 오류를 검출한다. 운영체제는 적절한 오류 메시지를 제공하고 비정상 종료시킬 수 있으며, 원문은 조사·수정을 위한 메모리 덤프도 설명한다.

그림 1.13 사용자 모드에서 커널 모드로의 전환

그림·표 출처: 책 26쪽 / 원문 PDF 30쪽

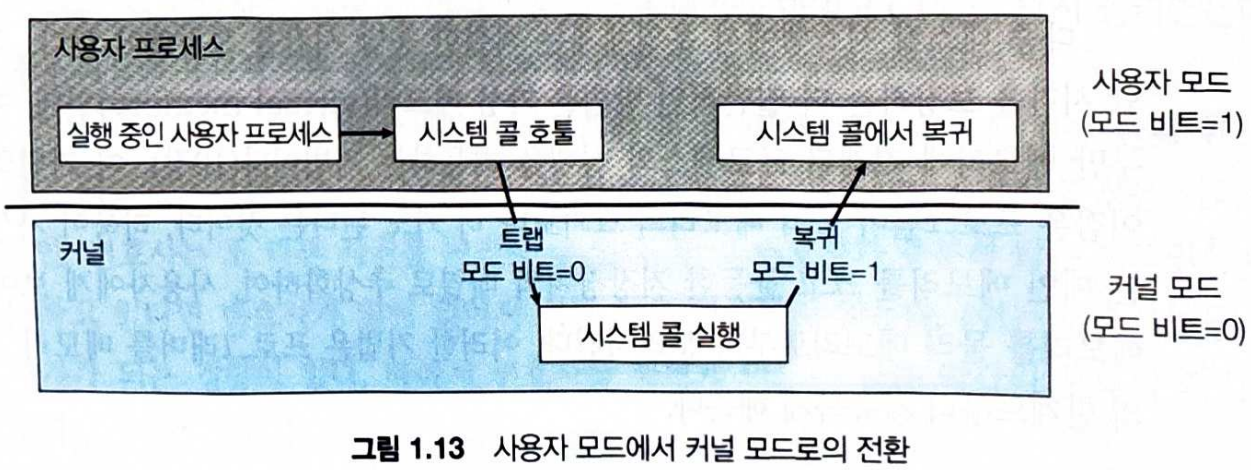


그림 1.13 사용자 모드에서 커널 모드로의 전환

그림 1.13 사용자 모드에서 커널 모드로의 전환

시스템 콜의 트랩에서 모드 비트가 0으로, 복귀에서 1로 바뀌는 원문 흐름이다.

1.4.3 타이머

출처: 책 28-29쪽 / 원문 PDF 32-33쪽

타이머는 사용자 프로그램의 무한 루프나 서비스 미호출 때문에 운영체제가 CPU 제어를 잃는 일을 방지한다. 지정한 시간 후 인터럽트를 발생시키며 가변 타이머는 고정률 클록과 계수기로 구현할 수 있다. 운영체제가 계수기를 설정하고 매 틱마다 감소시켜 0일 때 인터럽트를 발생시킨다. 제어를 사용자에게 넘기기 전에 설정되어야 하며 타이머 값 변경은 특권 명령이다. 원문 Linux 보충 설명에서 HZ는 초당 타이머 인터럽트 빈도, jiffies는 부팅 이후 인터럽트 누적 개수이다.

1.5.1 프로세스 관리

출처: 책 29-30쪽 / 원문 PDF 33-34쪽

프로세스는 실행 중인 프로그램의 인스턴스이며 CPU 시간·메모리·파일·입출력 장치와 초기화 데이터를 필요로 한다. 프로그램은 디스크에 놓인 수동적 개체이지만 프로세스는 다음 명령을 가리키는 프로그램 카운터를 갖는 능동적 개체이다. 동일 프로그램을 실행해도 별도의 프로세스가 될 수 있다. 단일 스레드에는 한 실행 순서가, 다중 스레드에는 각 스레드의 프로그램 카운터가 있다.

- 사용자·시스템 프로세스를 생성하고 제거한다.
- 프로세스와 스레드에 CPU를 스케줄하고 중지·재개한다.
- 프로세스 동기화와 통신을 위한 기법을 제공한다.
- 종료 시 재사용 가능한 자원을 회수한다.

1.5.2 메모리 관리

출처: 책 30-31쪽 / 원문 PDF 34-35쪽

메인 메모리는 CPU와 입출력 장치가 공유하는 바이트 배열이며 각 바이트에는 주소가 있다. 프로그램을 실행하려면 메모리에 적재하고 절대 주소로 매핑해야 한다. 여러 프로그램을 메모리에 유지하면 CPU 이용률과 응답성을 높일 수 있지만 메모리 관리가 필요하다. 운영체제는 사용 중인 영역과 소유 프로세스를 추적하고, 공간을 할당·회수하며, 적재하거나 제거할 프로세스 또는 그 일부를 결정한다. 관리 기법의 효율은 상황에 따라 달라지고 필요한 하드웨어 지원의 영향을 받는다.

1.5.3 파일 시스템 관리

출처: 책 31-32쪽 / 원문 PDF 35-36쪽

운영체제는 물리 저장 매체의 차이를 감추고 **파일**이라는 논리 저장 단위를 정의한다. 파일은 생성자가 정의한 관련 정보의 집합이며 프로그램과 데이터를 표현할 수 있다. 운영체제는 파일을 물리 매체로 매핑하고 디렉터리로 조직하며 사용자별 접근을 통제한다. 파일·디렉터리 생성과 제거, 조작 프리미티브, 보조저장장치 매핑, 안정적인 매체로의 백업이 책무이다.

1.5.4 대용량 저장장치 관리

출처: 책 32쪽 / 원문 PDF 36쪽

HDD·NVM 같은 보조저장장치는 적재 전 프로그램과 데이터의 주요 온라인 저장소이다. 운영체제는 마운팅·언마운팅, 가용 공간 관리, 저장장소 할당, 디스크 스케줄링, 분할과 보호를 맡는다. 매우 빈번하게 사용되므로 장치와 알고리즘의 효율이 전체 시스템 속도에 영향을 줄 수 있다. 백업·장기 보존용 3차 저장장치는 덜 빈번하게 쓰이더라도 장치 할당·반환과 데이터 이주 등의 관리가 필요하다.

1.5.5 캐시 관리

출처: 책 33-34쪽 / 원문 PDF 37-38쪽

캐싱은 정보를 더 빠른 저장장치에 일시 복사하는 원리이다. 먼저 캐시를 검사하여 있으면 직접 쓰고, 없으면 원래 저장장치에서 가져와 사용하면서 캐시에 둔다. 캐시는 제한된 크기이므로 크기와 교체 정책이 중요하다. CPU 레지스터는 메인 메모리의 고속 캐시처럼 볼 수 있으며 레지스터 할당은 프로그래머·컴파일러가 정한다. 하드웨어만으로 관리되는 캐시는 이 책의 운영체제 제어 대상과 구별한다.

한 데이터의 복사본이 디스크·메인 메모리·캐시·레지스터에 동시에 존재하면 값의 갱신 시점이 서로 달라진다. 다중 태스킹에서는 각 프로세스가 최신 값을 보도록 해야 한다. 다중 처리기에서는 여러 로컬 캐시의 복사본을 일치시키는 **캐시 일관성** 문제가 생기며 통상 하드웨어에서 처리한다. 분산 환경의 파일 복제본 갱신은 더욱 복잡하다.

1.5.6 입출력 시스템 관리

출처: 책 34-35쪽 / 원문 PDF 38-39쪽

입출력 서브시스템은 장치의 구체적인 특성을 사용자와 운영체제의 나머지 부분에서 숨긴다. 버퍼링·캐싱·스풀링을 포함하는 메모리 관리 부분, 일반 장치 드라이버 인터페이스, 특정 하드웨어 드라이버로 구성된다. 해당 장치의 특성을 직접 아는 것은 그 장치 드라이버이다.

1.6 보안과 보호

출처: 책 35-36쪽 / 원문 PDF 39-40쪽

개념	정의와 역할
보호(protection)	프로그램·프로세스·사용자의 자원 접근을 제어한다. 허가된 사용과 허가되지 않은 사용을 구분하고 제어 명세와 강제 시행 방법을 제공한다.
보안(security)	시스템 외부와 내부의 공격을 방어한다. 보호 기능이 작동해도 인증 정보가 도난되면 자료가 복사·삭제될 수 있다.

운영체제는 사용자 이름과 유일한 사용자 식별자를 연결하고 로그인 인증에서 이를 결정한다. 식별자는 사용자의 프로세스·스레드에 연관된다. 그룹 식별자는 여러 사용자에게 공통 접근 권한을 부여할 때 사용한다. 제한된 작업에 권한 상승이 필요할 수 있으며 원문은 UNIX setuid를 통해 파일 소유자의 유효 사용자 식별자로 실행하는 경우를 제시한다. 보호와 보안은 사용자를 구별하는 기반에 의존하지만 동일한 개념은 아니다.

1.7 가상화

출처: 책 36-38쪽 / 원문 PDF 40-42쪽

가상화는 한 컴퓨터의 하드웨어를 여러 실행 환경으로 추상화하여 각 환경이 독립된 컴퓨터에서 실행되는 듯하게 만드는 기술이다. VMM은 게스트 운영체제를 실행하고 자원 사용을 관리하며 게스트 사이를 보호한다. 원문은 호스트 운영체제 위의 응용 형태 VMM과 호스트 자체 역할을 하는 VMM을 모두 소개한다.

원문의 구별에서 에뮬레이션은 소스와 대상 CPU가 다를 때 하드웨어를 소프트웨어로 모의하고 명령을 번역하므로 비용이 크다. 가상화에서는 동일 CPU용으로 컴파일된 운영체제를 다른 운영체제 환경에서 실행한다. 활용 사례는 다른 운영체제의 응용 사용, 여러 플랫폼용 개발·시험·디버깅, 데이터 센터의 자원 관리이다.

1.8 분산 시스템

출처: 책 38-39쪽 / 원문 PDF 42-43쪽

분산 시스템은 물리적으로 떨어진 이기종 컴퓨터가 네트워크로 연결된 집합이다. 자원 공유는 계산 속도·기능·데이터 가용성·신뢰성을 높일 수 있다. 운영체제는 원격 접근을 파일 접근처럼 일반화하거나 네트워크 기능을 명시적으로 호출하게 한다. 네트워크는 프로토콜·거리·매체에 따라 다르며 원문은 LAN·WAN·MAN·PAN과 TCP/IP를 소개한다.

네트워크 운영체제의 컴퓨터는 다른 컴퓨터와 파일·메시지를 공유하지만 서로 자율적으로 동작한다. 분산 운영체제는 컴퓨터들이 밀접하게 통신하여 하나의 운영체제가 전체 네트워크를 제어하는 것처럼 보이게 하므로 자율성이 더 낮다.

1.9.1 리스트·스택·큐

출처: 책 39-41쪽 / 원문 PDF 43-45쪽

배열은 각 원소에 직접 접근한다. 리스트는 데이터를 순서로 나타내며 연결 리스트는 포인터로 항목을 연결한다. 단일 연결 리스트는 후속 항목, 이중 연결 리스트는 앞뒤 항목을 가리키고 원형 연결 리스트의 마지막은 첫 항목을 가리킨다. 가변 수 항목의 삽입·삭제는 쉽지만 길이 n인 리스트의 특정 항목을 찾는 최악 성능은 O(n)이다.

구조	순서·연산	원문의 사용
스택	후입선출(LIFO), push·pop	함수 인수·지역 변수·복귀 주소를 보관하고 복귀 시 제거한다.
큐	선입선출(FIFO)	인쇄 작업이나 CPU 실행 대기 태스크를 순서대로 조직한다.

1.9.2 트리

출처: 책 41쪽 / 원문 PDF 45쪽

트리는 부모·자식 관계로 계층을 나타낸다. 일반 트리는 임의 수의 자식, 이진 트리는 최대 두 자식을 가진다. 이진 탐색 트리는 값의 순서 관계를 추가하지만 형태에 따라 최악의 탐색이 O(n)이 될 수 있다. 균형 이진 탐색 트리는 깊이를 제한하여 최악의 탐색 O(lg n)을 보장한다. 원문은 Linux CPU 스케줄링의 red-black 트리를 예로 든다.

1.9.3 해시 함수와 맵

출처: 책 41-42쪽 / 원문 PDF 45-46쪽

해시 함수는 데이터에 산술 연산을 수행해 테이블 인덱스로 쓸 수 있는 수를 반환한다. 구현에 따라 인출 성능이 O(1)만큼 좋아질 수 있다. 서로 다른 입력이 같은 인덱스로 매핑되는 **해시 충돌**은 각 항목에 연결 리스트를 두어 처리할 수 있지만 충돌이 늘면 효율이 떨어진다. 해시 맵은 키와 값을 연결하며 키에 해시 함수를 적용해 값을 찾는다.

1.9.4 비트맵

출처: 책 42-43쪽 / 원문 PDF 46-47쪽

비트맵은 n개 자원의 상태를 n개 비트로 나타낸다. 원문의 예에서는 0이 사용 가능, 1이 사용 불가능을 뜻하지만 반대로 정할 수도 있다. i번째 비트는 i번째 자원에 대응한다. 한 자원에 8비트 부울 값을 쓰는 것보다 공간을 절약하므로 디스크 블록처럼 많은 자원의 가용성을 표시할 때 유용하다.

1.10.1 전통적 계산

출처: 책 43-44쪽 / 원문 PDF 47-48쪽

웹 포털·무선 연결·모바일 접근이 사무실과 가정의 전통적인 계산 환경 경계를 넓힌다. thin client는 웹 기반 계산을 이해하는 터미널로 보안과 유지보수 측면에서 워크스테이션을 대체하기도 한다. 배치 시스템은 미리 정해진 입력을 대량 처리하고 대화형 시스템은 사용자 입력을 기다린다. 시분할의 타이머·스케줄링 기법은 현대 워크스테이션과 서버에서도 프로세스에 계산 시간을 빈번하게 제공하는 데 사용된다.

1.10.2 모바일 컴퓨팅

출처: 책 44-45쪽 / 원문 PDF 48-49쪽

모바일 컴퓨팅은 가볍고 휴대 가능한 스마트폰·태블릿 환경이다. GPS·가속도계·자이로스코프를 활용하는 응용과 증강현실을 지원한다. 무선망이나 휴대 전화 데이터망으로 온라인 서비스에 접근한다. 원문은 PC에 비해 메모리·처리 능력이 제한되고 전력 소모가 중요한 설계 조건임을 설명하며 iOS와 Android를 예로 든다. 이 설명과 수치는 원문 집필 시점의 환경이다.

1.10.3 클라이언트·서버 컴퓨팅

출처: 책 45-46쪽 / 원문 PDF 49-50쪽

클라이언트가 생성한 요청을 서버가 충족시키는 분산 구조이다. 계산 서버는 작업 요청 인터페이스를 제공하고 수행 결과를 돌려준다. 파일 서버는 파일 생성·갱신·읽기·제거 인터페이스를 제공하며 웹 서버가 브라우저에 파일을 전달하는 경우가 예이다.

1.10.4 피어 간 계산

출처: 책 46-47쪽 / 원문 PDF 50-51쪽

피어 간(P2P) 시스템에서는 클라이언트와 서버가 고정적으로 구별되지 않고 각 노드가 요청·제공 상황에 따라 두 역할을 맡는다. 서비스가 여러 노드로 분산되어 전통적인 서버 병목을 줄일 수 있다. 서비스 발견에는 중앙 검색 서비스에 등록하는 방법과 요청을 방송하고 제공자가 응답하는 방법이 있다. 원문은 Napster·Gnutella·Skype로 이 구성을 설명한다.

1.10.5 클라우드 컴퓨팅

출처: 책 47-49쪽 / 원문 PDF 51-53쪽

클라우드 컴퓨팅은 계산·저장·응용을 네트워크 서비스로 제공한다. 가상화를 기반으로 이용할 수 있으며 사용자는 사용한 자원에 따라 비용을 지불할 수 있다. 공중·사유·혼합형이라는 운영 범위의 구별과 SaaS·PaaS·IaaS라는 제공 자원의 구별은 서로 독립적인 배타 분류가 아니다.

유형	원문 의미
공중·사유·혼합형	누구나 비용을 지불하고 사용하는 서비스 / 한 회사용 / 두 방식을 포함하는 방식
SaaS	인터넷으로 사용하는 응용
PaaS	응용에 맞게 준비된 소프트웨어 스택
IaaS	인터넷으로 사용하는 서버·저장장치

클라우드 하부구조에는 전통적 운영체제, 가상 머신을 관리하는 VMM, VMM과 클라우드 자원·인터페이스를 관리하는 도구가 결합될 수 있다. 원문 그림 1.24는 IaaS 공중 클라우드의 고객 인터페이스·관리 서비스·가상 머신·저장장치·부하 분배 관계를 보여 준다.

1.10.6 실시간 내장형 시스템

출처: 책 48-49쪽 / 원문 PDF 52-53쪽

내장형 시스템은 특정 작업을 수행하며 사용자 인터페이스가 거의 없을 수 있다. 범용 운영체제와 전용 응용, 제한된 기능의 내장형 운영체제, 운영체제 없이 ASIC만 사용하는 형태가 모두 존재한다. **실시간 시스템**은 데이터 흐름이나 처리에 엄격한 시간 제약이 있어 정해진 제한 시간 안에 올바른 결과를 내야 정확하게 동작한 것으로 간주된다. 단순히 빠른 응답이 바람직한 일반 시스템과 이 조건을 구별한다.

1.11 무료 및 공개 소스 운영체제

출처: 책 50쪽 / 원문 PDF 54쪽

무료·공개 소스 운영체제는 소스 코드 형태로 제공되어 분석·수정·컴파일·실행을 통한 학습을 가능하게 한다. 원문의 **자유 소프트웨어**는 소스 공개뿐 아니라 사용·재배포·수정을 허용하는 라이선스를 요구한다. 따라서 자유 소프트웨어는 공개 소스이지만 모든 공개 소스가 같은 자유를 보장하는 것은 아니다. 반면 비공개 소스는 컴파일된 이진 코드만 제공할 수 있다. 이진 코드에서 역공학으로 원래 소스와 주석을 완전히 복원하는 것은 어렵다.

1.11.1 역사

출처: 책 51쪽 / 원문 PDF 55쪽

초기에는 사용자 집단과 제조업체가 프로그램을 소스 코드와 함께 교환·배포했지만 소프트웨어의 사용 대상을 제한하고 경쟁자로부터 아이디어를 보호하려는 목적에서 이진 파일만 배포하는 관행이 확산되었다. 원문은 이러한 변화가 자유·공개 소스 운동의 배경임을 설명한다.

1.11.2 무료 운영체제

출처: 책 51-52쪽 / 원문 PDF 55-56쪽

Richard Stallman의 GNU와 자유 소프트웨어 운동에서 자유는 가격보다 사용자의 권리를 뜻한다. 실행, 분석·수정, 수정 없이 배포·판매, 수정해서 배포·판매할 자유가 핵심이다. **카피레프트**는 재배포 후에도 이러한 자유를 유지하게 한다. 원문은 GPL에서 바이너리와 함께 소스를 배포하고 수정본을 포함한 사본에 동일 GPL을 적용한다고 설명한다.

1.11.3 GNU/Linux

출처: 책 52-53쪽 / 원문 PDF 56-57쪽

GNU가 개발한 컴파일러·도구와 Linus Torvalds의 Linux 커널이 결합하여 GNU/Linux를 이룬다. 원문은 커널만을 Linux, GNU 도구까지 포함한 전체를 GNU/Linux로 구별한다. 배포판은 기능·유틸리티·응용·하드웨어 지원·인터페이스·목적에서 차이가 난다. 라이브 CD·DVD는 설치 없이 부팅하여 사용할 수 있고 가상 머신 이미지는 기존 호스트에서 별도의 운영체제를 학습하게 한다. 원문에 수록된 다운로드 주소와 설치 지시는 학습 사례로만 검토하였다.

1.11.4 BSD UNIX

출처: 책 53-54쪽 / 원문 PDF 57-58쪽

BSD UNIX는 AT&T UNIX에서 출발하였고 원문은 4.4BSD-lite 및 FreeBSD·NetBSD·OpenBSD·DragonFlyBSD를 소개한다. 소스 트리와 버전 제어를 이용하면 파일 변경 이력과 동시 수정 충돌을 관리할 수 있다. macOS의 공개 소스 핵심인 Darwin은 BSD UNIX에 기반하고 커널 패키지 이름은 xnu이다.

1.11.5 Solaris

출처: 책 54쪽 / 원문 PDF 58쪽

원문은 Solaris의 BSD UNIX와 System V 기반 이력, OpenSolaris 공개, 그 기반에서 발전한 Illumos를 설명한다. 이 이력은 제공된 본문의 설명 범위이며 이후 프로젝트의 현황을 외부 자료로 보충하지 않는다.

1.11.6 학습 도구로서 공개 소스 시스템

출처: 책 54-55쪽 / 원문 PDF 58-59쪽

소스가 공개되면 동작 관찰이나 문헌만으로 추측하던 구현을 직접 검토할 수 있다. 프로그램을 수정·시험하고 다양한 목표·라이선스·구현을 비교할 수 있다. 가상화와 하드웨어 시뮬레이터는 전용 기계 없이 여러 운영체제나 역사적 시스템을 탐구하게 한다. 원문은 소스 공개의 학습 가치와 다양성을 강조한다.

1.12 요약과 원문 복습 범위

출처: 책 56-59쪽 / 원문 PDF 60-63쪽

원문 요약은 하드웨어 관리, 인터럽트, 저장 계층, 다중 프로그래밍·태스킹, 실행 모드, 프로세스·메모리·파일 관리, 보호·가상화·계산 환경·공개 소스를 연결한다. 장말 연습문제 1.1~1.11은 운영체제 목적, 실시간 제약, 구성 범위, 특권 명령, 캐시, 클라이언트·서버와 P2P를 점검한다. 추가 자료와 참고문헌은 존재와 주제를 검토했으며 그 외부 문헌의 내용이나 새 풀이를 포함하지 않는다.

핵심 정리

- 운영체제의 자원 할당과 제어 역할을 사용자 관점과 시스템 관점에 연결한다.
- 인터럽트는 상태 보존·핸들러 디스패치·복원으로 처리하며 DMA는 블록 전송의 CPU 개입을 줄인다.
- 다중 프로그래밍은 CPU 이용, 다중 태스킹은 빈번한 전환과 응답에 초점을 둔다.
- 커널 모드·특권 명령·타이머는 잘못된 프로그램으로부터 시스템 제어를 지킨다.
- 보호는 자원 접근 제어, 보안은 내부·외부 공격 방어이다.
- 자원 복사본이 늘면 최신 값과 일관성을 유지하는 문제가 생긴다.

혼동하기 쉬운 내용

다중 프로그래밍과 다중 태스킹

출처: 책 24-26쪽 / 원문 PDF 28-30쪽

항목	다중 프로그래밍	다중 태스킹
목적	실행할 일이 있으면 CPU를 계속 사용	프로세스를 빈번하게 전환하여 빠른 응답
관계	여러 프로그램을 메모리에 유지	다중 프로그래밍의 논리적 확장

장 마무리 복습

- 하드웨어 자원을 추상화·할당·보호하는 것이 프로그램 실행 환경의 토대이다.
- CPU 제어의 이동은 인터럽트·트랩·시스템 콜 및 실행 모드 전환과 연결된다.
- 단일 처리기·다중 처리기·클러스터는 범용 계산 단위와 결합 방식의 차이로 구별한다.
- 저장 계층은 속도·용량·비용·휘발성의 절충이며 캐시에는 크기·교체·일관성 문제가 있다.

제2장. 운영체제 구조

출처: 원문 PDF 1쪽

학습 주제

- 제공 파일의 목차에는 제2장이 있지만 해당 본문은 제공되지 않았다. 본문 지식은 원문에서 확인되지 않음.

2.1 운영체제 서비스 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.2 사용자와 운영체제 인터페이스 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.3 시스템 콜 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.4 시스템 서비스 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.5 링커와 로더 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.6 응용 프로그램이 운영체제마다 다른 이유 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.7 운영체제 설계 및 구현 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.8 운영체제 구조 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.9 운영체제 빌딩과 부팅 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.10 운영체제 디버깅 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

2.11 요약 (목차만 확인)

출처: 원문 PDF 1쪽

이 절의 제목만 목차에서 확인된다. 본문 내용은 원문에서 확인되지 않음.

핵심 정리

- 제2장 본문은 원문에서 확인되지 않음.

원문 확인 메모

- 목차는 인쇄 61~112쪽의 장·절을 안내하지만 실제 파일은 인쇄 59쪽 다음에 117쪽으로 넘어간다. 본문 내용을 목차 제목에서 추정하지 않았다.

제7부 도입. 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

학습 주제

- 제공된 마지막 도입 페이지에서 보안과 보호의 역할을 구별한다.

P7.1 도입에서 확인되는 보안과 보호의 역할

출처: 책 679쪽 / 원문 PDF 615쪽

보안은 저장된 자료와 코드의 무결성뿐 아니라 물리 자원의 보호를 위해 사용자 인증을 보장한다. 인가되지 않은 접근, 접근 후의 악의적인 파괴·자료 변경, 우연한 비일관성 도입을 막는 역할을 한다.

보호는 사용자의 파일 접근 유형을 제한해 제어된 접근을 제공한다. 운영체제에서 적절한 권한을 얻은 프로세스만 메모리 세그먼트·CPU 등의 자원에 작업할 수 있도록 한다. 프로그램·프로세스·사용자의 자원 접근을 제어하는 기법에는 부과할 제어를 지정하는 방법과 시행하는 방법이 필요하다.

이 페이지는 제7부의 도입이다. 뒤의 제16장 보안과 제17장 보호 본문은 제공되지 않아, 구체적인 위협·암호 기법·접근 행렬 등의 설명은 원문에서 확인되지 않음으로 남긴다.

핵심 정리

- 보안은 사용자 인증과 인가되지 않은 접근·자료 훼손의 방지를 다룬다.
- 보호는 권한에 따른 자원 접근 제어를 다룬다. 제어를 지정하고 시행하는 방법이 필요하다.

혼동하기 쉬운 내용

도입에서 구별한 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

개념	도입에 제시된 역할
보안	사용자 인증, 무결성과 물리 자원 보호, 인가되지 않은 접근과 파괴·변경 방지
보호	파일 접근 유형 제한과 권한에 따른 자원 접근 제어, 제어의 지정과 시행

장 마무리 복습

- 보안과 보호의 역할 구별은 제공된 도입의 범위에서만 복습한다.

원문 확인 메모

- 제16~21장 및 부록 본문은 미제공이다. 제7부 도입만으로 해당 장의 내용을 보충하지 않았다.

비교·공식·절차 모음

제1장. 서론

다중 프로그래밍과 다중 태스킹

출처: 책 24-26쪽 / 원문 PDF 28-30쪽

항목	다중 프로그래밍	다중 태스킹
목적	실행할 일이 있으면 CPU를 계속 사용	프로세스를 빈번하게 전환하여 빠른 응답
관계	여러 프로그램을 메모리에 유지	다중 프로그래밍의 논리적 확장

제7부 도입. 보안과 보호

도입에서 구별한 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

개념	도입에 제시된 역할
보안	사용자 인증, 무결성과 물리 자원 보호, 인가되지 않은 접근과 파괴·변경 방지
보호	파일 접근 유형 제한과 권한에 따른 자원 접근 제어, 제어의 지정과 시행

주요 공식과 기호

주요 처리 순서

제1장. 서론 / 1.2.1

출처: 책 7-12쪽 / 원문 PDF 11-16쪽

인터럽트 처리의 기본 순서

1. CPU는 명령어 실행을 마칠 때 인터럽트 요청 라인을 검사한다.

↓

2. 인터럽트 번호를 인터럽트 벡터의 인덱스로 사용해 해당 핸들러로 이동한다.

↓

3. 핸들러는 변경될 상태를 저장하고 원인을 확인하여 필요한 처리를 수행한다.

↓

4. 상태를 복원하고 인터럽트 복귀 명령으로 이전 실행을 재개한다.

알고리즘·원문 코드 찾아보기

제1장. 서론

코드·그림	원문 출처	교재 쪽
그림 1.6 저장장치 계층 구조	출처: 책 14쪽 / 원문 PDF 18쪽	8
그림 1.13 사용자 모드에서 커널 모드로의 전환	출처: 책 26쪽 / 원문 PDF 30쪽	11

주요 개념 복습

제1장. 서론

출처: 책 3-59쪽 / 원문 PDF 7-63쪽

- 하드웨어 자원을 추상화·할당·보호하는 것이 프로그램 실행 환경의 토대이다.
- CPU 제어의 이동은 인터럽트·트랩·시스템 콜 및 실행 모드 전환과 연결된다.
- 단일 처리기·다중 처리기·클러스터는 범용 계산 단위와 결합 방식의 차이로 구별한다.
- 저장 계층은 속도·용량·비용·휘발성의 절충이며 캐시에는 크기·교체·일관성 문제가 있다.

제2장. 운영체제 구조

출처: 원문 PDF 1쪽

- 제2장 본문은 원문에서 확인되지 않음.

제7부 도입. 보안과 보호

출처: 책 679쪽 / 원문 PDF 615쪽

- 보안과 보호의 역할 구별은 제공된 도입의 범위에서만 복습한다.

개념 색인

개념	교재 쪽	개념	교재 쪽
DMA	9	NUMA	9
SMP	9	가상화	13
다중 태스킹	10	다중 프로그래밍	10
보안 / 보호의 도입	19	비트맵	14
실시간 시스템	15	인터럽트	7
캐시 일관성	12	특권 명령	10

최종 학습 점검표

제1장. 서론

[ ] 인터럽트 발생부터 상태 복원까지의 순서를 설명할 수 있는가?

출처: 책 7-12쪽 / 원문 PDF 11-16쪽

[ ] 다중 프로그래밍과 다중 태스킹의 목적을 구별할 수 있는가?

출처: 책 24-26쪽 / 원문 PDF 28-30쪽

[ ] 사용자·커널 모드와 타이머의 보호 역할을 설명할 수 있는가?

출처: 책 26-29쪽 / 원문 PDF 30-33쪽

[ ] 보호와 보안, 가상화와 에뮬레이션을 구별할 수 있는가?

출처: 책 35-38쪽 / 원문 PDF 39-42쪽

[ ] 리스트·스택·큐·트리·맵·비트맵의 원문 용도를 설명할 수 있는가?

출처: 책 39-43쪽 / 원문 PDF 43-47쪽

제7부 도입. 보안과 보호

[ ] 제공된 도입에 따라 보안과 보호의 역할을 구별할 수 있는가?

출처: 책 679쪽 / 원문 PDF 615쪽